

- AI驱动测试任务管理系统 - 技术文档
 - 目录
 - 1. 系统架构概述
 - 1.1 整体架构
 - 1.2 服务职责划分
 - 2. 技术栈详解
 - 2.1 前端技术栈
 - 2.2 后端技术栈
 - 2.3 AI服务技术栈
 - 3. AI核心技术实现
 - 3.1 智能测试用例生成
 - 3.1.1 技术方案: RAG + LLM
 - 3.1.2 实现细节
 - 3.1.3 RAG检索实现
 - 3.1.4 Prompt工程
 - 3.1.5 LLM模型选择
 - 3.2 策略智能推荐
 - 3.2.1 技术方案: XGBoost + 规则引擎
 - 3.2.2 实现细节
 - 3.2.3 特征工程
 - 3.2.4 规则引擎
 - 3.2.5 XGBoost模型训练
 - 3.3 智能去重
 - 3.3.1 技术方案: Sentence-BERT语义相似度
 - 3.3.2 实现细节
 - 3.3.3 Sentence-BERT模型
 - 3.4 优先级智能排序
 - 3.4.1 技术方案: 多因子加权评分
 - 3.4.2 实现细节
 - 3.4.3 权重配置
 - 3.5 质量智能分析
 - 3.5.1 技术方案: 多维度质量评估
 - 3.5.2 实现细节
 - 3.5.3 执行质量分析
 - 3.5.4 覆盖质量分析
 - 3.5.5 改进建议生成
 - 4. 数据库设计

- 4.1 MySQL (业务数据)
 - 4.2 Qdrant (向量数据库)
 - 4.3 Redis (缓存)
- 5. API接口设计
 - 5.1 AI服务API
 - 5.1.1 测试用例生成
 - 5.1.2 策略推荐
 - 5.1.3 用例去重
 - 5.1.4 优先级排序
 - 5.1.5 质量分析
 - 5.2 后端服务API
 - 5.2.1 测试任务管理
 - 5.2.2 测试用例管理
- 6. 性能优化策略
 - 6.1 AI推理优化
 - 6.2 向量检索优化
 - 6.3 数据库优化
 - 6.4 缓存策略
- 7. 安全性设计
 - 7.1 认证与授权
 - 7.2 数据安全
 - 7.3 Prompt注入防护
- 附录
 - A. 技术选型对比
 - B. 性能基准
 - C. 依赖版本

AI驱动测试任务管理系统 - 技术文档

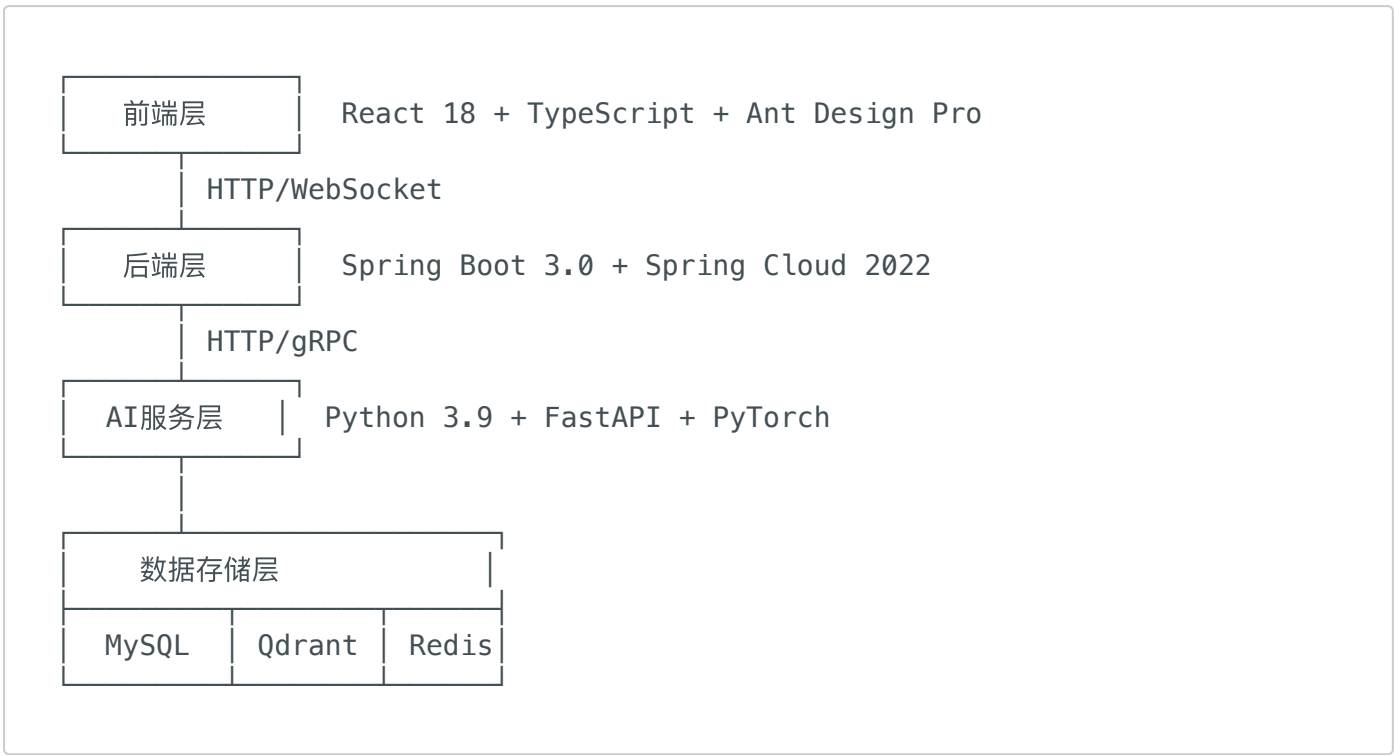
目录

- 系统架构概述
- 技术栈详解
- AI核心技术实现
- 数据库设计
- API接口设计

1. 系统架构概述

1.1 整体架构

系统采用微服务架构，由三个主要服务组成：



1.2 服务职责划分

• Frontend (前端服务)

- 用户交互界面
- 数据可视化展示
- 实时状态监控

• Backend (后端服务)

- 业务逻辑处理
- 权限认证授权
- 数据持久化管理

- 任务调度与执行

- **AI Service (AI服务)**

- 测试用例智能生成
- 策略智能推荐
- 用例智能去重
- 优先级智能排序
- 质量智能分析

2. 技术栈详解

2.1 前端技术栈

技术	版本	用途
React	18.x	UI框架
TypeScript	5.x	类型安全
Ant Design Pro	6.x	UI组件库
Redux Toolkit	2.x	状态管理
Vite	5.x	构建工具
ECharts	5.x	数据可视化
Axios	1.x	HTTP客户端

2.2 后端技术栈

技术	版本	用途
Spring Boot	3.0.x	应用框架
Spring Cloud	2022.x	微服务框架
Java	11	编程语言
Maven	3.8.x	依赖管理

技术	版本	用途
MySQL	8.0	业务数据存储
Redis	7.x	缓存
Kafka	7.4.x	消息队列

2.3 AI服务技术栈

技术	版本	用途
Python	3.9	编程语言
FastAPI	0.100+	Web框架
Uvicorn	0.23+	ASGI服务器
PyTorch	2.0+	深度学习框架
Transformers	4.30+	NLP模型库
Sentence-Transformers	2.2+	句向量模型
Scikit-learn	1.3+	机器学习库
XGBoost	2.0+	梯度提升框架
Qdrant	1.7.4	向量数据库
MySQL	8.0	结构化数据存储

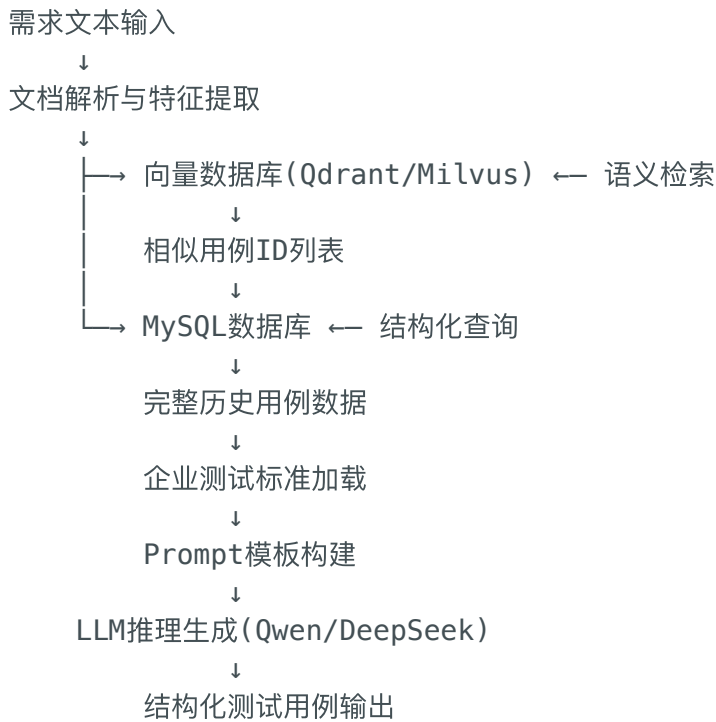
3. AI核心技术实现

3.1 智能测试用例生成

3.1.1 技术方案：RAG + LLM

核心思路: 结合检索增强生成(Retrieval Augmented Generation)和大语言模型(Large Language Model)，实现高质量测试用例的智能生成。

技术架构:



3.1.2 实现细节

文件位置: `ai-service/models/llm/rag_generator.py`

核心类: `RAGTestCaseGenerator`

生成流程:

```
def generate(
    self,
    requirement_text: str,
    module: str,
    num_cases: int = 5,
    include_edge_cases: bool = True
) -> Dict[str, Any]:
    """
    智能生成测试用例

    参数:
        requirement_text: 需求描述文本
        module: 所属模块
        num_cases: 生成用例数量
        include_edge_cases: 是否包含边界用例

    返回:
        生成的测试用例列表及元数据
    """

    # 步骤1: 解析需求文档, 提取关键信息
    parsed_req = self._parse_requirement(requirement_text)
```

```

# 步骤2: RAG检索相似历史用例
similar_cases = self.rag_manager.search_similar_testcases(
    query_text=requirement_text,
    top_k=5,
    module_filter=module
)

# 步骤3: 加载企业测试标准
standards = self._load_company_standards(module)

# 步骤4: 构建Prompt
prompt = self._build_prompt(
    requirement=parsed_req,
    similar_cases=similar_cases,
    standards=standards,
    num_cases=num_cases,
    include_edge_cases=include_edge_cases
)

# 步骤5: 调用LLM生成
response = self.llm_client.generate(
    prompt=prompt,
    max_tokens=2000,
    temperature=0.7
)

# 步骤6: 解析并结构化输出
testcases = self._parse_llm_response(response)

return {
    "testcases": testcases,
    "metadata": {
        "requirement_id": parsed_req.get("id"),
        "module": module,
        "generation_time": datetime.now().isoformat(),
        "model_used": self.llm_client.model_name
    }
}

```

3.1.3 RAG检索实现

文件位置: `ai-service/data/rag_manager.py`

核心类: `RAGManager`

混合检索策略:

```

def search_similar_testcases(
    self,
    query_text: str,
    top_k: int = 5,
    module_filter: Optional[str] = None

```

```

) -> List[Dict[str, Any]]:
    """
    混合检索：向量数据库 + MySQL

    步骤1：向量数据库语义检索（Qdrant/Milvus）
    步骤2：MySQL获取完整数据
    """

    # 1. 文本向量化
    query_embedding = self.embedding_model.encode(query_text)

    # 2. 向量数据库检索（Qdrant）
    search_results = self.vector_db_client.search(
        collection_name="testcases",
        query_vector=query_embedding,
        limit=top_k,
        filter={
            "module": module_filter
        } if module_filter else None
    )

    # 3. 提取候选ID
    candidate_ids = [hit.id for hit in search_results]

    # 4. MySQL批量查询完整数据
    full_testcases = self.mysql_client.query(
        """
        SELECT id, title, description, precondition,
               steps, expected_result, priority, module
        FROM testcases
        WHERE id IN (%s)
        ORDER BY FIELD(id, %s)
        """ % (','.join(['%s'] * len(candidate_ids)),
              ','.join(['%s'] * len(candidate_ids))),
        candidate_ids + candidate_ids
    )

    # 5. 合并相似度分数
    for i, testcase in enumerate(full_testcases):
        testcase['similarity_score'] = search_results[i].score

    return full_testcases

```

3.1.4 Prompt工程

Prompt模板结构:

```

PROMPT_TEMPLATE = """
# 角色定义
你是一位资深的软件测试专家，擅长编写高质量的测试用例。

# 任务
根据以下需求描述，生成 {num_cases} 个测试用例。

```



```
# 需求描述
{requirement_text}

# 参考历史相似用例
{similar_cases_context}

# 企业测试标准
{company_standards}

# 输出要求
1. 用例应覆盖正常场景、异常场景和边界场景
2. 每个用例必须包含：
    - 用例标题 (title)
    - 用例描述 (description)
    - 前置条件 (precondition)
    - 测试步骤 (steps, 数组格式)
    - 预期结果 (expected_result)
    - 优先级 (priority: P0/P1/P2/P3)
3. 输出格式为JSON数组

# 生成用例
""""
```

3.1.5 LLM模型选择

模型	参数量	优势	使用场景
Qwen-7B-Chat	7B	中文理解强，推理速度快	通用测试用例生成
DeepSeek-Coder	6.7B	代码理解能力强	接口/单元测试用例
Llama-3-8B	8B	多语言支持	国际化项目

模型配置:

```
# ai-service/models/llm/llm_client.py
class LLMClient:
    def __init__(self, provider: str = "qwen"):
        self.provider = provider

        if provider == "qwen":
            self.model_name = "Qwen/Qwen-7B-Chat"
            self.max_length = 8192
        elif provider == "deepseek":
            self.model_name = "deepseek-ai/deepseek-coder-6.7b-instruct"
            self.max_length = 16384

        self.model = AutoModelForCausalLM.from_pretrained(
            self.model_name,
            torch_dtype=torch.float16,
            device_map="auto"
```

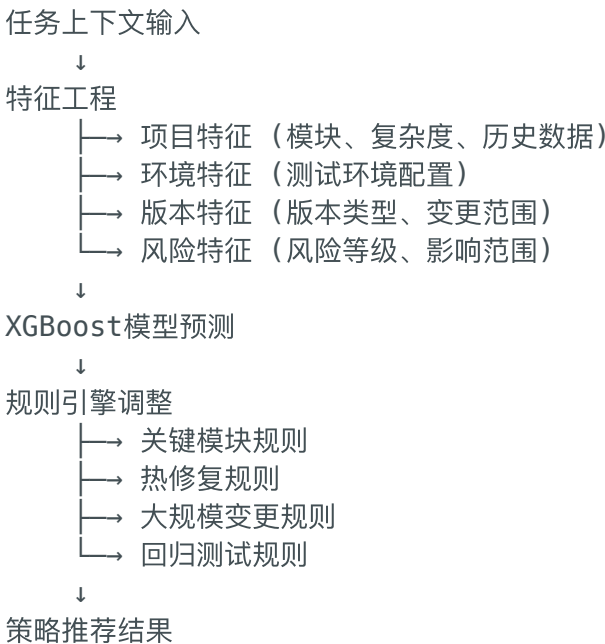
```
)  
self.tokenizer = AutoTokenizer.from_pretrained(self.model_name)
```

3.2 策略智能推荐

3.2.1 技术方案：XGBoost + 规则引擎

核心思路: 结合机器学习模型和业务规则，实现测试策略的智能推荐。

技术架构:



3.2.2 实现细节

文件位置: `ai-service/models/recommendation/strategy_recommender.py`

核心类: `TestStrategyRecommender`

推荐流程:

```
def recommend(self, context: Dict[str, Any]) -> Dict[str, Any]:  
    """  
    智能推荐测试策略  
  
    参数:  
        context: 任务上下文信息  
            - module: 模块名称
```

- version_type: 版本类型 (major/minor/patch/hotfix)
- change_scope: 变更范围 (small/medium/large)
- risk_level: 风险等级 (low/medium/high/critical)
- test_environment: 测试环境

返回:

推荐的测试策略

"""

步骤1: 特征提取

features = self._extract_features(context)

步骤2: ML模型预测

if self.model:

ml_prediction = self._predict_with_model(features)

else:

ml_prediction = self._heuristic_prediction(features)

步骤3: 规则引擎调整

adjusted_strategy = self.rule_engine.apply_rules(

prediction=ml_prediction,

context=context

)

步骤4: 生成详细建议

recommendations = self._generate_recommendations(

strategy=adjusted_strategy,

context=context

)

return {

"strategy": adjusted_strategy,

"recommendations": recommendations,

"confidence": ml_prediction.get("confidence", 0.85)

}

3.2.3 特征工程

```
def _extract_features(self, context: Dict[str, Any]) -> np.ndarray:
    """
    提取特征向量
    """
    features = []

    # 1. 模块复杂度特征
    module = context.get("module", "")
    module_complexity = self._get_module_complexity(module)
    features.append(module_complexity)

    # 2. 历史失败率
    historical_failure_rate = self._get_historical_failure_rate(module)
    features.append(historical_failure_rate)

    # 3. 变更规模编码
```

```

        change_scope_map = {"small": 0.3, "medium": 0.6, "large": 1.0}
        change_scope = change_scope_map.get(context.get("change_scope",
"medium"), 0.6)
        features.append(change_scope)

    # 4. 风险等级编码
    risk_level_map = {"low": 0.25, "medium": 0.5, "high": 0.75, "critical":
1.0}
    risk_level = risk_level_map.get(context.get("risk_level", "medium"),
0.5)
    features.append(risk_level)

    # 5. 版本类型编码
    version_type_map = {"patch": 0.2, "minor": 0.5, "major": 0.8, "hotfix":
1.0}
    version_type = version_type_map.get(context.get("version_type",
"minor"), 0.5)
    features.append(version_type)

    # 6. 测试覆盖率
    current_coverage = context.get("current_coverage", 0.7)
    features.append(current_coverage)

    # 7. 代码变更行数（归一化）
    lines_changed = min(context.get("lines_changed", 100) / 1000.0, 1.0)
    features.append(lines_changed)

    # 8. 依赖模块数量（归一化）
    dependencies = min(context.get("dependencies_count", 5) / 20.0, 1.0)
    features.append(dependencies)

    return np.array(features).reshape(1, -1)

```

3.2.4 规则引擎

文件位置: `ai-service/models/recommendation/strategy_recommender.py`

核心类: `RuleEngine`

```

class RuleEngine:
    """
    业务规则引擎
    """

    def apply_rules(
        self,
        prediction: Dict[str, Any],
        context: Dict[str, Any]
    ) -> Dict[str, Any]:
        """
        应用业务规则调整策略
        """
        strategy = prediction.copy()

```

```

# 规则1: 关键模块必须全量回归
if context.get("is_critical_module", False):
    strategy["test_scope"] = "full_regression"
    strategy["priority"] = "P0"

# 规则2: 热修复必须快速验证
if context.get("version_type") == "hotfix":
    strategy["test_scope"] = "targeted"
    strategy["execution_mode"] = "parallel"
    strategy["smoke_test_required"] = True

# 规则3: 大规模变更增加测试范围
if context.get("change_scope") == "large":
    if strategy["test_scope"] == "smoke":
        strategy["test_scope"] = "partial_regression"
    strategy["additional_checks"] = ["integration", "performance"]

# 规则4: 高风险必须人工审核
if context.get("risk_level") in ["high", "critical"]:
    strategy["manual_review_required"] = True
    strategy["approval_required"] = True

# 规则5: 生产环境部署前必须UAT
if context.get("target_environment") == "production":
    strategy["uat_required"] = True
    strategy["rollback_plan_required"] = True

return strategy

```

3.2.5 XGBoost模型训练

```

# ai-service/models/recommendation/train_strategy_model.py
def train_strategy_model(training_data_path: str):
    """
    训练测试策略推荐模型
    """
    # 1. 加载训练数据
    df = pd.read_csv(training_data_path)

    # 2. 特征和标签分离
    X = df.drop(['strategy_label'], axis=1)
    y = df['strategy_label']

    # 3. 划分训练集和测试集
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # 4. XGBoost参数配置
    params = {
        'objective': 'multi:softmax',
        'num_class': 4, # smoke, partial, full, targeted
        'max_depth': 6,

```

```
'learning_rate': 0.1,
'n_estimators': 100,
'subsample': 0.8,
'colsample_bytree': 0.8,
'random_state': 42
}

# 5. 训练模型
model = xgb.XGBClassifier(**params)
model.fit(X_train, y_train)

# 6. 模型评估
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.4f}")

# 7. 保存模型
model.save_model('models/strategy_recommender.json')

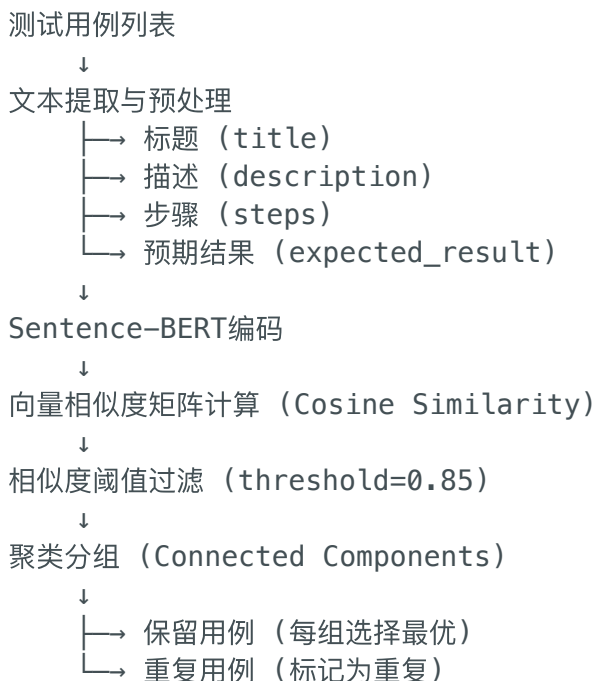
return model
```

3.3 智能去重

3.3.1 技术方案：Sentence-BERT语义相似度

核心思路: 使用预训练的Sentence-BERT模型计算测试用例之间的语义相似度，通过聚类算法识别重复用例。

技术架构:



3.3.2 实现细节

文件位置: `ai-service/models/optimization/deduplicator.py`

核心类: `SemanticDeduplicator`

去重流程:

```
class SemanticDeduplicator:
    """
    基于语义相似度的智能去重
    """

    def __init__(
        self,
        model_name: str = "paraphrase-multilingual-mpnet-base-v2",
        similarity_threshold: float = 0.85
    ):
        """
        初始化

        参数:
            model_name: Sentence-BERT模型名称
            similarity_threshold: 相似度阈值 (0.85表示85%相似即视为重复)
        """
        self.model = SentenceTransformer(model_name)
        self.similarity_threshold = similarity_threshold

    def deduplicate(
        self,
        testcases: List[Dict[str, Any]]
    ) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]:
        """
        执行智能去重

        返回:
            (保留的用例列表, 重复的用例列表)
        """

        # 步骤1: 文本提取
        texts = self._extract_texts(testcases)

        # 步骤2: 计算向量表示
        embeddings = self.model.encode(texts, convert_to_tensor=True)

        # 步骤3: 计算相似度矩阵
        similarity_matrix = util.cos_sim(embeddings, embeddings)

        # 步骤4: 识别重复组
        duplicate_groups = self._find_duplicate_groups(
            similarity_matrix,
            self.similarity_threshold
```

```

    )

    # 步骤5: 选择保留用例
    kept_testcases, duplicate_testcases = self._select_best_from_groups(
        testcases,
        duplicate_groups
    )

    return kept_testcases, duplicate_testcases

def _extract_texts(self, testcases: List[Dict[str, Any]]) -> List[str]:
    """
    提取测试用例文本
    """
    texts = []
    for tc in testcases:
        # 拼接关键字段
        text_parts = [
            tc.get("title", ""),
            tc.get("description", ""),
            " ".join(tc.get("steps", [])),
            tc.get("expected_result", "")
        ]
        text = " ".join(filter(None, text_parts))
        texts.append(text)
    return texts

def _find_duplicate_groups(
    self,
    similarity_matrix: torch.Tensor,
    threshold: float
) -> List[List[int]]:
    """
    使用连通分量算法找出重复组
    """
    n = similarity_matrix.shape[0]

    # 构建邻接矩阵 (相似度超过阈值的视为连通)
    adj_matrix = (similarity_matrix > threshold).cpu().numpy()

    # 避免自连接
    np.fill_diagonal(adj_matrix, False)

    # 使用DFS找出所有连通分量
    visited = [False] * n
    groups = []

    def dfs(node, group):
        visited[node] = True
        group.append(node)
        for neighbor in range(n):
            if adj_matrix[node][neighbor] and not visited[neighbor]:
                dfs(neighbor, group)

    for i in range(n):
        if not visited[i]:
            group = []

```



```

        dfs(i, group)
        if len(group) > 1: # 只关注有重复的组
            groups.append(group)

    return groups

def _select_best_from_groups(
    self,
    testcases: List[Dict[str, Any]],
    duplicate_groups: List[List[int]]
) -> Tuple[List[Dict[str, Any]], List[Dict[str, Any]]]:
    """
    从每个重复组中选择最优用例

    选择策略：
    1. 优先级最高 (P0 > P1 > P2 > P3)
    2. 步骤最详细 (步骤数量最多)
    3. 创建时间最早 (更成熟)
    """
    kept_indices = set(range(len(testcases)))
    duplicate_testcases = []

    for group in duplicate_groups:
        # 评分排序
        scores = []
        for idx in group:
            tc = testcases[idx]
            score = self._calculate_quality_score(tc)
            scores.append((idx, score))

        # 按分数降序排序
        scores.sort(key=lambda x: x[1], reverse=True)

        # 保留最优, 其余标记为重复
        best_idx = scores[0][0]
        for idx, _ in scores[1:]:
            kept_indices.discard(idx)
            tc_copy = testcases[idx].copy()
            tc_copy['duplicate_of'] = testcases[best_idx].get('id')
            duplicate_testcases.append(tc_copy)

    kept_testcases = [testcases[i] for i in sorted(kept_indices)]

    return kept_testcases, duplicate_testcases

def _calculate_quality_score(self, testcase: Dict[str, Any]) -> float:
    """
    计算测试用例质量分数
    """
    score = 0.0

    # 优先级权重
    priority_weights = {"P0": 4.0, "P1": 3.0, "P2": 2.0, "P3": 1.0}
    score += priority_weights.get(testcase.get("priority", "P2"), 2.0) *

```

```
steps_count = len(testcase.get("steps", []))
score += min(steps_count * 2, 20)

# 描述完整性
if testcase.get("description"):
    score += 10
if testcase.get("precondition"):
    score += 5
if testcase.get("expected_result"):
    score += 10

# 执行历史（有执行记录的更可靠）
if testcase.get("execution_count", 0) > 0:
    score += 15

return score
```

3.3.3 Sentence-BERT模型

模型选择: `paraphrase-multilingual-mpnet-base-v2`

模型特点:

- 支持50+种语言，包括中文
- 768维向量表示
- 在语义相似度任务上表现优异
- 推理速度快 (每秒可处理数千个句子)

向量维度: 768

相似度计算: 余弦相似度 (Cosine Similarity)

```
# 余弦相似度公式
cosine_similarity = (A · B) / (||A|| * ||B||)

# 其中:
# A, B: 两个向量
# A · B: 向量点积
# ||A||, ||B||: 向量的L2范数
```

3.4 优先级智能排序

3.4.1 技术方案: 多因子加权评分

核心思路: 综合考虑业务价值、风险等级、执行成本、历史失败率、覆盖范围等多个维度，计算测试用例的优先级分数。

评分模型:

```
优先级分数 =  
    业务价值 × 0.30 +  
    风险等级 × 0.25 +  
    执行成本 × 0.20 +  
    历史失败率 × 0.15 +  
    覆盖影响 × 0.10
```

3.4.2 实现细节

文件位置: `ai-service/models/optimization/prioritizer.py`

核心类: `TestCasePrioritizer`

排序流程:

```
class TestCasePrioritizer:  
    """  
    测试用例智能优先级排序  
    """  
  
    def __init__(self, weights: Optional[Dict[str, float]] = None):  
        """  
        初始化  
  
        参数:  
            weights: 各因子权重配置  
        """  
        self.weights = weights or {  
            "business_value": 0.30,  
            "risk_level": 0.25,  
            "execution_cost": 0.20,  
            "failure_history": 0.15,  
            "coverage_impact": 0.10  
        }  
  
    def prioritize(  
        self,  
        testcases: List[Dict[str, Any]]  
    ) -> List[Dict[str, Any]]:  
        """  
        执行智能排序  
  
        返回:  
            按优先级降序排列的测试用例列表  
        """
```

```

"""

# 步骤1: 计算每个用例的优先级分数
for tc in testcases:
    tc['priority_score'] = self._calculate_priority_score(tc)

# 步骤2: 按分数降序排序
sorted_testcases = sorted(
    testcases,
    key=lambda x: x['priority_score'],
    reverse=True
)

# 步骤3: 分配执行顺序
for idx, tc in enumerate(sorted_testcases, start=1):
    tc['execution_order'] = idx

return sorted_testcases

def _calculate_priority_score(self, testcase: Dict[str, Any]) -> float:
    """
    计算优先级分数 (0-100)
    """

    # 因子1: 业务价值 (0-100)
    business_value = self._calculate_business_value(testcase)

    # 因子2: 风险等级 (0-100)
    risk_level = self._calculate_risk_level(testcase)

    # 因子3: 执行成本 (0-100, 成本越低分数越高)
    execution_cost = self._calculate_execution_cost(testcase)

    # 因子4: 历史失败率 (0-100)
    failure_history = self._calculate_failure_history(testcase)

    # 因子5: 覆盖影响 (0-100)
    coverage_impact = self._calculate_coverage_impact(testcase)

    # 加权求和
    score = (
        business_value * self.weights["business_value"] +
        risk_level * self.weights["risk_level"] +
        execution_cost * self.weights["execution_cost"] +
        failure_history * self.weights["failure_history"] +
        coverage_impact * self.weights["coverage_impact"]
    )

    return round(score, 2)

def _calculate_business_value(self, testcase: Dict[str, Any]) -> float:
    """
    计算业务价值分数

    考虑因素:
    - 用例优先级标签 (P0/P1/P2/P3)
    - 关联功能的业务重要性
    """

```

```

- 用户使用频率
"""
score = 0.0

# 优先级标签映射
priority_map = {
    "P0": 100, # 核心功能
    "P1": 75,  # 重要功能
    "P2": 50,  # 一般功能
    "P3": 25   # 次要功能
}
priority = testcase.get("priority", "P2")
score += priority_map.get(priority, 50)

# 用户使用频率调整
usage_frequency = testcase.get("usage_frequency", "medium")
if usage_frequency == "high":
    score *= 1.2
elif usage_frequency == "low":
    score *= 0.8

return min(score, 100)

def _calculate_risk_level(self, testcase: Dict[str, Any]) -> float:
    """
    计算风险等级分数

    考虑因素：
    - 代码变更影响范围
    - 模块稳定性
    - 历史问题严重程度
    """
    risk_map = {
        "critical": 100,
        "high": 80,
        "medium": 50,
        "low": 20
    }

    risk_level = testcase.get("risk_level", "medium")
    return risk_map.get(risk_level, 50)

def _calculate_execution_cost(self, testcase: Dict[str, Any]) -> float:
    """
    计算执行成本分数（成本越低，分数越高）

    考虑因素：
    - 执行时间
    - 环境依赖复杂度
    - 人工介入程度
    """
    # 执行时间（秒）
    execution_time = testcase.get("estimated_duration", 60)

    # 时间越短，分数越高（假设最长1小时）
    time_score = max(0, 100 - (execution_time / 36))

```

```

# 自动化程度
is_automated = testcase.get("is_automated", False)
automation_bonus = 20 if is_automated else 0

score = time_score + automation_bonus
return min(score, 100)

def _calculate_failure_history(self, testcase: Dict[str, Any]) -> float:
    """
    计算历史失败率分数

    考虑因素：
    - 历史执行次数
    - 历史失败次数
    - 最近失败时间
    """
    execution_count = testcase.get("execution_count", 0)
    failure_count = testcase.get("failure_count", 0)

    if execution_count == 0:
        return 30 # 未执行过的用例给予中等分数

    failure_rate = failure_count / execution_count

    # 失败率越高，分数越高（这些用例更需要关注）
    score = failure_rate * 100

    # 最近失败加成
    last_failure = testcase.get("last_failure_time")
    if last_failure:
        days_since_failure = (datetime.now() -
                               datetime.fromisoformat(last_failure)).days
        if days_since_failure <= 7:
            score *= 1.3 # 最近失败过，提高优先级

    return min(score, 100)

def _calculate_coverage_impact(self, testcase: Dict[str, Any]) -> float:
    """
    计算覆盖影响分数

    考虑因素：
    - 代码覆盖范围
    - 功能覆盖广度
    - 依赖关系数量
    """
    score = 50 # 基础分

    # 覆盖的代码行数
    code_lines_covered = testcase.get("code_lines_covered", 0)
    score += min(code_lines_covered / 10, 30)

    # 依赖的模块数量
    dependencies_count = testcase.get("dependencies_count", 0)
    score += min(dependencies_count * 5, 20)

    return min(score, 100)

```

3.4.3 权重配置

不同场景下可以调整权重配置:

场景1: 快速回归 (追求速度)

```
weights = {  
    "business_value": 0.40, # 提高业务价值权重  
    "risk_level": 0.30,    # 提高风险权重  
    "execution_cost": 0.20, # 关注执行成本  
    "failure_history": 0.05,  
    "coverage_impact": 0.05  
}
```

场景2: 全面测试 (追求覆盖)

```
weights = {  
    "business_value": 0.20,  
    "risk_level": 0.20,  
    "execution_cost": 0.10,  
    "failure_history": 0.20,  
    "coverage_impact": 0.30 # 大幅提高覆盖权重  
}
```

场景3: 问题排查 (追求稳定性)

```
weights = {  
    "business_value": 0.15,  
    "risk_level": 0.35,    # 提高风险权重  
    "execution_cost": 0.10,  
    "failure_history": 0.35, # 大幅提高历史失败权重  
    "coverage_impact": 0.05  
}
```

3.5 质量智能分析

3.5.1 技术方案：多维度质量评估

核心思路: 从执行结果、代码覆盖、缺陷分布、测试完整性等多个维度，综合评估测试质量。

分析维度:



3.5.2 实现细节

文件位置: ai-service/services/analysis_service.py

核心函数: analyze_test_quality

```
def analyze_test_quality(
    task_id: str,
    analysis_period: str = "latest"
) -> Dict[str, Any]:
    """
    执行测试质量分析

    参数:
        task_id: 测试任务ID
        analysis_period: 分析周期 (latest/7d/30d)

    返回:
        质量分析报告
    """

    # 步骤1: 获取测试执行数据
    execution_data = fetch_execution_data(task_id, analysis_period)

    # 步骤2: 执行质量分析
    execution_quality = analyze_execution_quality(execution_data)
```



```

# 步骤3: 覆盖质量分析
coverage_quality = analyze_coverage_quality(execution_data)

# 步骤4: 缺陷质量分析
defect_quality = analyze_defect_quality(execution_data)

# 步骤5: 完整性质量分析
completeness_quality = analyze_completeness_quality(execution_data)

# 步骤6: 计算综合质量分数
overall_score = (
    execution_quality["score"] * 0.40 +
    coverage_quality["score"] * 0.30 +
    defect_quality["score"] * 0.20 +
    completeness_quality["score"] * 0.10
)

# 步骤7: 生成改进建议
recommendations = generate_recommendations({
    "execution": execution_quality,
    "coverage": coverage_quality,
    "defect": defect_quality,
    "completeness": completeness_quality
})

# 步骤8: 趋势分析
trend_analysis = analyze_quality_trend(task_id)

return {
    "task_id": task_id,
    "overall_score": round(overall_score, 2),
    "grade": get_quality_grade(overall_score),
    "dimensions": {
        "execution": execution_quality,
        "coverage": coverage_quality,
        "defect": defect_quality,
        "completeness": completeness_quality
    },
    "recommendations": recommendations,
    "trend": trend_analysis,
    "generated_at": datetime.now().isoformat()
}

```

3.5.3 执行质量分析

```

def analyze_execution_quality(execution_data: Dict) -> Dict[str, Any]:
    """
    分析执行质量
    """
    total = execution_data["total_cases"]
    passed = execution_data["passed_cases"]
    failed = execution_data["failed_cases"]
    blocked = execution_data["blocked_cases"]

```

```

# 计算各项指标
pass_rate = (passed / total * 100) if total > 0 else 0
fail_rate = (failed / total * 100) if total > 0 else 0
block_rate = (blocked / total * 100) if total > 0 else 0

# 计算执行效率
avg_execution_time = execution_data.get("avg_execution_time", 0)

# 评分逻辑
score = pass_rate * 0.7 # 通过率占70%

# 失败率惩罚
if fail_rate > 20:
    score -= (fail_rate - 20) * 0.5

# 阻塞率惩罚
if block_rate > 10:
    score -= (block_rate - 10) * 1.0

score = max(0, min(score, 100))

return {
    "score": round(score, 2),
    "metrics": {
        "pass_rate": round(pass_rate, 2),
        "fail_rate": round(fail_rate, 2),
        "block_rate": round(block_rate, 2),
        "avg_execution_time": avg_execution_time
    },
    "status": "excellent" if score >= 90 else "good" if score >= 70 else
"warning" if score >= 50 else "poor"
}

```

3.5.4 覆盖质量分析

```

def analyze_coverage_quality(execution_data: Dict) -> Dict[str, Any]:
    """
    分析覆盖质量
    """
    code_coverage = execution_data.get("code_coverage", 0)
    function_coverage = execution_data.get("function_coverage", 0)
    branch_coverage = execution_data.get("branch_coverage", 0)

    # 覆盖率加权平均
    coverage_score = (
        code_coverage * 0.4 +
        function_coverage * 0.3 +
        branch_coverage * 0.3
    )

    # 场景覆盖评估
    covered_scenarios = execution_data.get("covered_scenarios", [])
    total_scenarios = execution_data.get("total_scenarios", 1)
    scenario_coverage = len(covered_scenarios) / total_scenarios * 100

```

```

# 综合评分
score = (coverage_score * 0.7 + scenario_coverage * 0.3)

return {
    "score": round(score, 2),
    "metrics": {
        "code_coverage": round(code_coverage, 2),
        "function_coverage": round(function_coverage, 2),
        "branch_coverage": round(branch_coverage, 2),
        "scenario_coverage": round(scenario_coverage, 2)
    },
    "uncovered_areas": identify_uncovered_areas(execution_data),
    "status": "excellent" if score >= 80 else "good" if score >= 60 else
"warning" if score >= 40 else "poor"
}

```

3.5.5 改进建议生成

```

def generate_recommendations(quality_data: Dict) -> List[Dict[str, str]]:
    """
    基于质量分析结果生成改进建议
    """
    recommendations = []

    # 执行质量建议
    exec_quality = quality_data["execution"]
    if exec_quality["metrics"]["fail_rate"] > 15:
        recommendations.append({
            "priority": "high",
            "category": "execution",
            "title": "失败率过高",
            "description": f"当前失败率为 {exec_quality['metrics']
['fail_rate']:.2f}%, 建议分析失败原因并优化用例。",
            "actions": [
                "分析Top失败用例的根因",
                "检查测试环境稳定性",
                "优化不稳定的测试用例"
            ]
        })

    if exec_quality["metrics"]["block_rate"] > 5:
        recommendations.append({
            "priority": "medium",
            "category": "execution",
            "title": "阻塞率偏高",
            "description": f"当前阻塞率为 {exec_quality['metrics']
['block_rate']:.2f}%, 建议排查环境和依赖问题。",
            "actions": [
                "检查测试环境配置",
                "验证依赖服务可用性",
                "优化用例前置条件"
            ]
        })

```

```

# 覆盖质量建议
cov_quality = quality_data["coverage"]
if cov_quality["score"] < 60:
    recommendations.append({
        "priority": "high",
        "category": "coverage",
        "title": "覆盖率不足",
        "description": f"当前综合覆盖率仅 {cov_quality['score']:.2f}分，建议
增加测试用例。",
        "actions": [
            "针对未覆盖区域补充用例",
            "增加边界场景测试",
            "补充异常流程测试"
        ]
    })

# 缺陷质量建议
defect_quality = quality_data["defect"]
if defect_quality["metrics"]["defect_density"] > 10:
    recommendations.append({
        "priority": "critical",
        "category": "defect",
        "title": "缺陷密度过高",
        "description": f"每千行代码缺陷数为 {defect_quality['metrics']
['defect_density']:.2f}，建议进行代码质量审查。",
        "actions": [
            "进行代码审查",
            "加强单元测试",
            "重构高缺陷密度模块"
        ]
    })

# 完整性建议
comp_quality = quality_data["completeness"]
if comp_quality["score"] < 70:
    recommendations.append({
        "priority": "medium",
        "category": "completeness",
        "title": "测试完整性不足",
        "description": "部分用例缺少必要信息或追溯关系。",
        "actions": [
            "完善用例描述和步骤",
            "补充需求追溯关系",
            "更新测试文档"
        ]
    })

return recommendations

```

4. 数据库设计

4.1 MySQL (业务数据)

主要表结构:

-- 测试用例表

```
CREATE TABLE testcases (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  title VARCHAR(255) NOT NULL,  
  description TEXT,  
  precondition TEXT,  
  steps JSON,  
  expected_result TEXT,  
  priority ENUM('P0', 'P1', 'P2', 'P3') DEFAULT 'P2',  
  module VARCHAR(100),  
  created_by BIGINT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  INDEX idx_module (module),  
  INDEX idx_priority (priority)  
);
```

-- 测试任务表

```
CREATE TABLE test_tasks (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(255) NOT NULL,  
  description TEXT,  
  module VARCHAR(100),  
  version VARCHAR(50),  
  environment VARCHAR(50),  
  status ENUM('pending', 'running', 'completed', 'cancelled') DEFAULT  
  'pending',  
  created_by BIGINT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  INDEX idx_status (status),  
  INDEX idx_created_at (created_at)  
);
```

-- 执行记录表

```
CREATE TABLE execution_records (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  task_id BIGINT NOT NULL,  
  testcase_id BIGINT NOT NULL,  
  status ENUM('passed', 'failed', 'blocked', 'skipped') NOT NULL,  
  execution_time INT,  
  error_message TEXT,  
  executed_by BIGINT,  
  executed_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (task_id) REFERENCES test_tasks(id),  
  FOREIGN KEY (testcase_id) REFERENCES testcases(id),  
  INDEX idx_task_id (task_id),  
  INDEX idx_status (status)  
);
```

-- 缺陷表

```
CREATE TABLE defects (  
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  
    title VARCHAR(255) NOT NULL,  
    description TEXT,  
    severity ENUM('critical', 'high', 'medium', 'low') NOT NULL,  
    status ENUM('open', 'in_progress', 'resolved', 'closed') DEFAULT 'open',  
    testcase_id BIGINT,  
    task_id BIGINT,  
    created_by BIGINT,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (testcase_id) REFERENCES testcases(id),  
    INDEX idx_severity (severity),  
    INDEX idx_status (status)  
);
```

4.2 Qdrant (向量数据库)

集合配置:

```
# 创建测试用例向量集合  
from qdrant_client import QdrantClient  
from qdrant_client.models import Distance, VectorParams  
  
client = QdrantClient(host="localhost", port=6333)  
  
client.create_collection(  
    collection_name="testcases",  
    vectors_config=VectorParams(  
        size=768, # Sentence-BERT向量维度  
        distance=Distance.COSINE # 余弦相似度  
    )  
)  
  
# 插入向量数据  
from qdrant_client.models import PointStruct  
  
client.upsert(  
    collection_name="testcases",  
    points=[  
        PointStruct(  
            id=testcase_id,  
            vector=embedding_vector, # 768维向量  
            payload={  
                "title": "...",  
                "module": "...",  
                "priority": "..."  
            }  
        )  
    ]  
)
```

4.3 Redis (缓存)

缓存策略:

```
# 1. 用例生成结果缓存 (TTL: 1小时)
redis_client.setex(
    key=f"generation:{request_hash}",
    time=3600,
    value=json.dumps(result)
)

# 2. 相似用例检索缓存 (TTL: 30分钟)
redis_client.setex(
    key=f"similar:{query_hash}",
    time=1800,
    value=json.dumps(similar_cases)
)

# 3. 策略推荐缓存 (TTL: 15分钟)
redis_client.setex(
    key=f"recommendation:{context_hash}",
    time=900,
    value=json.dumps(recommendation)
)

# 4. 用户会话缓存 (TTL: 2小时)
redis_client.setex(
    key=f"session:{user_id}",
    time=7200,
    value=json.dumps(session_data)
)
```

5. API接口设计

5.1 AI服务API

基础路径: <http://ai-service:8000/api/v1>

5.1.1 测试用例生成

POST /testcase/generate
Content-Type: application/json

```
{
  "requirement_text": "用户登录功能需求描述...",
  "module": "user-auth",
  "num_cases": 5,
  "include_edge_cases": true,
  "generation_mode": "rag"
}
```

Response 200:

```
{
  "code": 0,
  "message": "success",
  "data": {
    "testcases": [
      {
        "title": "正常用户名密码登录",
        "description": "...",
        "precondition": "...",
        "steps": ["...", "..."],
        "expected_result": "...",
        "priority": "P0"
      }
    ],
    "metadata": {
      "generation_time": "2024-01-15T10:30:00",
      "model_used": "Qwen-7B-Chat"
    }
  }
}
```

5.1.2 策略推荐

POST /recommendation/strategy
Content-Type: application/json

```
{
  "module": "payment",
  "version_type": "minor",
  "change_scope": "medium",
  "risk_level": "high",
  "test_environment": "staging"
}
```

Response 200:

```
{
  "code": 0,
  "data": {
    "strategy": {
      "test_scope": "partial_regression",

```



```
    "execution_mode": "parallel",
    "priority": "P1"
  },
  "recommendations": [
    {
      "category": "scope",
      "suggestion": "建议执行支付模块及其依赖模块的测试用例"
    }
  ],
  "confidence": 0.87
}
```

5.1.3 用例去重

POST /testcase/deduplicate
Content-Type: application/json

```
{
  "testcases": [
    {
      "id": 1,
      "title": "...",
      "description": "...",
      "steps": ["..."]
    }
  ],
  "similarity_threshold": 0.85
}
```

Response 200:

```
{
  "code": 0,
  "data": {
    "kept_testcases": [...],
    "duplicate_testcases": [
      {
        "id": 5,
        "duplicate_of": 1,
        "similarity_score": 0.92
      }
    ],
    "summary": {
      "original_count": 100,
      "duplicate_count": 15,
      "kept_count": 85
    }
  }
}
```

5.1.4 优先级排序

POST /testcase/prioritize
Content-Type: application/json

```
{
  "testcases": [...],
  "weights": {
    "business_value": 0.30,
    "risk_level": 0.25,
    "execution_cost": 0.20,
    "failure_history": 0.15,
    "coverage_impact": 0.10
  }
}
```

Response 200:

```
{
  "code": 0,
  "data": {
    "sorted_testcases": [
      {
        "id": 1,
        "priority_score": 87.5,
        "execution_order": 1
      }
    ]
  }
}
```

5.1.5 质量分析

POST /analysis/quality
Content-Type: application/json

```
{
  "task_id": 12345,
  "analysis_period": "latest"
}
```

Response 200:

```
{
  "code": 0,
  "data": {
    "overall_score": 78.5,
    "grade": "good",
    "dimensions": {
      "execution": {...},
      "coverage": {...},
      "defect": {...},
      "completeness": {...}
    },
    "recommendations": [...]
  }
}
```

```
}  
}
```

5.2 后端服务API

基础路径: <http://backend:8080/api/v1>

5.2.1 测试任务管理

```
POST /task/create  
GET /task/list  
GET /task/{id}  
PUT /task/{id}  
DELETE /task/{id}  
POST /task/{id}/execute  
GET /task/{id}/status
```

5.2.2 测试用例管理

```
POST /testcase/create  
GET /testcase/list  
GET /testcase/{id}  
PUT /testcase/{id}  
DELETE /testcase/{id}  
POST /testcase/batch-import
```

6. 性能优化策略

6.1 AI推理优化

- 模型量化**: 使用INT8量化减少模型大小和推理时间
- 批处理**: 批量处理测试用例生成请求
- GPU加速**: 使用CUDA加速Transformer模型推理
- 模型缓存**: 缓存模型加载结果, 避免重复加载

```
# 模型量化示例
from transformers import AutoModelForCausalLM
import torch

model = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen-7B-Chat",
    torch_dtype=torch.float16, # FP16量化
    device_map="auto",
    load_in_8bit=True # INT8量化
)
```

6.2 向量检索优化

1. **HNSW索引**: Qdrant使用HNSW算法加速向量检索
2. **分片策略**: 按模块分片存储向量数据
3. **预过滤**: 使用payload过滤减少检索范围

```
# Qdrant检索优化
search_result = client.search(
    collection_name="testcases",
    query_vector=embedding,
    limit=5,
    filter={
        "must": [
            {"key": "module", "match": {"value": "payment"}},
            {"key": "priority", "match": {"any": ["P0", "P1"]}}
        ]
    },
    search_params={
        "hnsw_ef": 128, # 提高检索精度
        "exact": False # 使用近似检索
    }
)
```

6.3 数据库优化

1. **索引优化**: 为高频查询字段添加索引
2. **读写分离**: 使用MySQL主从复制
3. **连接池**: 配置合理的数据库连接池大小

```
# MySQL连接池配置
from sqlalchemy import create_engine
from sqlalchemy.pool import QueuePool
```

```
engine = create_engine(
    "mysql+pymysql://user:password@localhost:3306/testdb",
    poolclass=QueuePool,
    pool_size=20,          # 连接池大小
    max_overflow=40,       # 最大溢出连接数
    pool_timeout=30,       # 获取连接超时时间
    pool_recycle=3600      # 连接回收时间
)
```

6.4 缓存策略

1. 多级缓存: 本地缓存 + Redis缓存
2. 缓存预热: 系统启动时预加载热点数据
3. 缓存更新: 使用消息队列异步更新缓存

7. 安全性设计

7.1 认证与授权

- **JWT Token:** 用户身份认证
- **RBAC:** 基于角色的访问控制
- **API密钥:** 服务间通信认证

7.2 数据安全

- **敏感数据加密:** AES-256加密存储
- **SQL注入防护:** 使用参数化查询
- **XSS防护:** 输入输出过滤

7.3 Prompt注入防护

```
def sanitize_user_input(text: str) -> str:
    """
    防止Prompt注入攻击
    """
```

```
# 移除特殊指令标记
forbidden_patterns = [
    r"<|.|.*?\|>",          # 特殊标记
    r"system:",              # 系统指令
    r"assistant:",          # 助手指令
    r"###\s*Instruction",    # 指令注入
]

for pattern in forbidden_patterns:
    text = re.sub(pattern, "", text, flags=re.IGNORECASE)

# 长度限制
if len(text) > 10000:
    text = text[:10000]

return text.strip()
```

附录

A. 技术选型对比

组件类型	选项1	选项2	选型结果	理由
向量数据库	Qdrant	Milvus	Qdrant	ARM64支持更好，部署简单
LLM模型	Qwen-7B	Llama-3-8B	Qwen-7B	中文理解能力更强
句向量模型	paraphrase-multilingual	multilingual-e5	paraphrase-multilingual	多语言支持更成熟
ML框架	XGBoost	LightGBM	XGBoost	社区生态更完善

B. 性能基准

操作	响应时间	QPS
测试用例生成 (5个)	~5s	12
RAG相似检索	~100ms	500

操作	响应时间	QPS
语义去重 (100个)	~800ms	75
优先级排序 (100个)	~50ms	1000
策略推荐	~200ms	250

C. 依赖版本

完整依赖请参考:

- AI Service: [ai-service/requirements.txt](#)
- Backend: [backend/pom.xml](#)
- Frontend: [frontend/package.json](#)